

How to implement Notes with SAPscript DataProvider and Richtext

Created by Zink, Rene on Nov 21, 2025 • 7 minute read

- Overview
- Implement EXIT-Class
- Create a notetype
- Create an assignment
- Define SAPscript(ITF) to HTML mappings
- Delegate actions
 - Activate
 - Edit
 - Delete/Discard
- Integrate frontend
- CustomizingObject

Overview

To get an overview of the Notes Reuse Component please read the documentation. This is only a short description how to integrate Notes in your application.

Implement EXIT-Class

Notes acts as a subcomponent of the application object. It is needed to implement an EXIT-Class which includes the interface `if_gbt_nte_dp_exit` and to implement the methods.

-  In case you **do not** want to use notes with SAPscript DataProvider and Richtext please have a look at the other "How to implement Notes...".
-  External DataProvider usage is to access and manipulate these documents in Fiori UI, If you use SAPscript, texts can be accessed directly via SAPscript-API and when you use GOS-Notes it can be accessed by `GOS-API CL_GOS_API`.

```
cl_gbt_nte_demo_exit
```

```
interfaces if_gbt_nte_dp_exit.
```

It is necessary that the followed method are implemented at your exit class.

```
cl_gbt_nte_demo_exit
```

```
method if_gbt_nte_exit~create_instance.  
* here you return an instance of the object  
  if go_instance is initial.  
    create object go_instance.  
  endif.  
  ro_instance ?= go_instance.  
endmethod.
```

```
cl_gbt_nte_demo_exit
```

```
method if_gbt_nte_exit~authority_check.  
* here the application related authority check should be executed.  
  if should_be_editable = if_gbt_nte_exit~auth_edit.  
    ev_authority = if_gbt_nte_exit~auth_edit.  
  elseif should_be_display_onnly = if_gbt_nte_exit~auth_display_only.  
    ev_authority = if_gbt_nte_exit~auth_display_only  
  elseif should_be_no_permissions = if_gbt_nte_exit~auth_no_permission.  
    ev_authority = if_gbt_nte_exit~auth_no_permission.  
  endif.  
endmethod.
```

For more information please have a look at the section "2.1.6 Authority Check" of the documentation.

```
cl_gbt_nte_demo_exit
```

```
method if_gbt_nte_exit~is_note_read_only.  
* here special cases of read-only restrictions can be implemented. With rv_read_only = abap_true, the note is not enabled for editing.  
  if is_note-notetype = 'NOTETYPE1' and is_note-creator ne sy-uname.  
    rv_read_only = abap_true.  
  endif.  
endmethod.
```

In addition it is necessary to implement followed methods for the integration of an external Data Provider.

```
cl_gbt_nte_demo_exit
```

```
method if_gbt_nte_dp_exit~get_item_keys.  
* determine all text keys which should be displayed for this object and the given import parameters
```

```

*   iv_objectnodetype, iv_objectid, iv_notetype.
*   ...
   ls_key = value gty_item_key_struc(
       tdoobject = 'ExampleOBJECT'
       tdname    = 'ExampleTDNAME'
       tdid      = 'ExampleID'
       tdstyle   = 'ExampleTEXTSTYLE'
   ).
   append value gty_item_key_struc( item_key = ls_key ) to et_keys.
*   ...
endmethod.

```

cl_gbt_nte_demo_exit

```

method if_gbt_nte_dp_exit~get_key_for_new_item.
*   create and return new key for a new note which is created by user.
*   the new note will be stored afterwards with this key.
*   ...
   ls_key = value gty_item_key_struc(
       tdoobject = 'ExampleOBJECT'
       tdname    = 'ExampleTDNAME'
       tdid      = 'ExampleID'
       tdstyle   = 'ExampleTEXTSTYLE'
   ).
   rs_key = ls_key.
endmethod.

```

cl_gbt_nte_demo_exit

```

method if_gbt_nte_dp_exit~put_changed_keys.
*   this method informs the application about the changes after activation.
*   if required update the application tables with the new or deleted keys for the given object.
   if lt_deletes is not initial.
       delete from_application_tables from table lt_deletes.
   endif.

   if lt_inserts is not initial.
       insert into_application_tables from table lt_inserts.
   endif.

endmethod.

```

For more information please have a look at the section "2.1.7.2 Interface IF_GBT_NTE_DP_EXIT" of the documentation and the class documentation of CL_GBT_NTE_DP_SCR.

Create a notetype

Go to transaction SM30 and open the maintenance view VSGBT_NTE_TYPE. Create a notetype and maintain the attributes.

To use SAPscript with Richtext you should enter the Data Provider Class CL_GBT_NTE_DP_SCR and the Profile should be "individual formatted Text" RTE_CUSTOM.

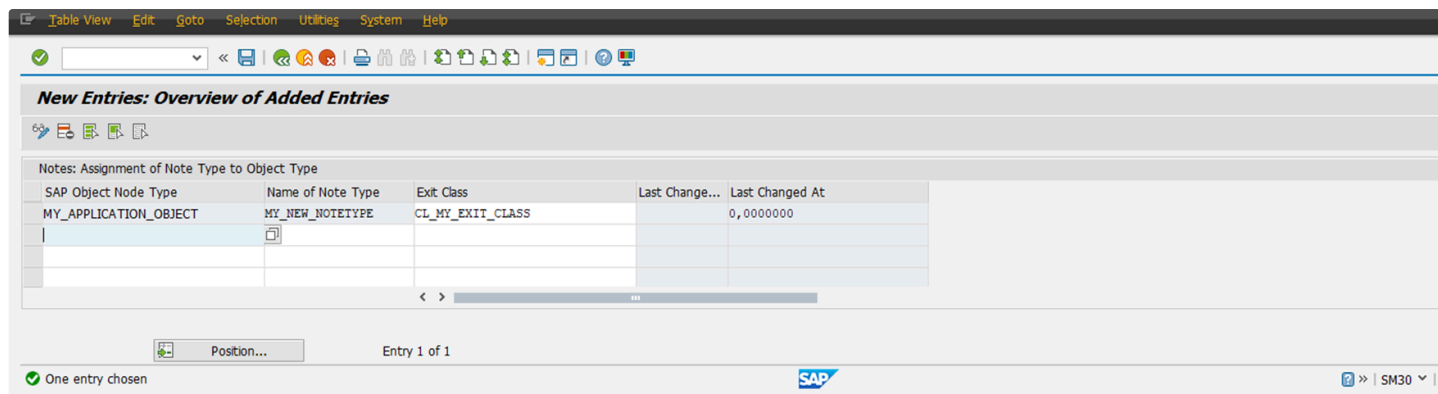
Name of Note Type	Lang-Dep.	Maximum Length	Multiple	No subject	Last Change...	Last Changed At	Profile for Note Format	Data Provider Class	Description of Note Type
MY_NEW_NOTETYPE	☒	3.000	☐	☐		0,0000000	individual formatted Text	CL_GBT_NTE_DP_SCR	My notes
	☐		☐	☐			Not defined		
	☐		☐	☐			Not defined		
	☐		☐	☐			Not defined		

For more information please have a look at the section "2.1.2 Note Type" and "2.1.7 Data Provider" of the documentation and the class documentation of CL_GBT_NTE_DP_SCR.

i In case of "individual formatted Text" by default all formatting options are disabled and should be enabled if required. See Section "Customizing Object".

Create an assignment

Go to transaction SM30 and open maintenance view VSGBT_NTE_TYASGN. Here you define the relation between object node type and note type. In addition, you provide your EXIT-Class which includes the interface. This class is called for the authority checks.



For more information please have a look at the section "2.2.1 Table Maintenance View" of the documentation.

Define SAPscript(ITF) to HTML mappings

Go to transaction SM30 and open the maintenance view VSGBT_NTE_SCRHTM.

In this view the application can maintain mappings to enable the format conversion from SAPscript(ITF) to html. It is required to map every formatting of sapscript to the right html tag and if requierd html style.

i In case that no mapping is defined for a formatting this formatting get lost!

To define a mapping the followed fields must be defined.

Column	Description	Example
Style (TDSTYLE)	SAPscript style for which the mapping should be relevant	'DOKU', 'S_OFFICE'
Format (TDFORMAT)	SAPscript format which should be mapped	'U' - undeline, 'H1' - heading
Mappingtype (MAP_TYP)	SAPscript distinguish between paragraph and character formatting. Paragraph formatting is like Heading or Left/Right/Block alignment it's relevant for the complete row (In ITF the first two characters of a line) Character formatting is like italic/bold/underline which is relevant for some characters (In ITF similar to HTML with an Tag like this '<U>formatted characters</>').	'P' - Paragraph mapping 'C' - Character mapping
Mappingpriority (MAP_PRIO)	With the priority the application gets the possibility to prioritize mappings. In certain circumstances it will be required to map HTML-Tag and Styles with priority to a specific ITF-Format or in case that its not fit exactly to define a fallback. Priority 1 has the highest priority.	'1' - '99'
HTML Tag (HTML_TAG)	HTML Tag which should be handled and converted. Here only the tag name is entered and not the complete Tag with opening and closing angle brackets.	'p' - 'span' - 'h1' - 'em' -
HTML Tag Style (HTML_STYLE)	It can be defined additional HTML Tag Style informations to a HTML Tag which will be added to a HTML Tag durring ITF to HTML conversion. In case of HTML to ITF conversion the converter searches for these tags with the specific style informations and the formatting will be converted to ITF.	'text-decoration: underline' -

TODO: add table with examples

Delegate actions

To realize a transactional orchestration between the application and the notesreuse component an integration from backend side is necessary.

The action ACTIVATE, EDIT, DISCARD and DELETE must be delegated (Reuse Components).

Activate

In your BO you have to delegate the ACTIVATE action to notes.

```
cl_i_dr_sgbt_nte_cds_demoth

method /bobf/if_frw_draft~copy_draft_to_active_entity.
* ...
* -----
*      Begin of integration of the notes reuse component to active existing drafts
* -----
      data(lo_nte_draft_mgr) = cl_gbt_nte_actn_delgatn_mgr=>create_instance( iv_objectnodelype = lr_root_draft->objectnodelype
                                                                    iv_objectid      = lv_final_objectid
                                                                    iv_draftuuid      = lr_root_draft->key "DraftUUID of application object
                                                                    it_notetypes      = lt_ntypes ).

      lo_nte_draft_mgr->activate( importing et_failed_key = et_failed_draft_key
```

```

                                changing co_message = eo_message ).
*-----
*      End of integration of the notes reuse component to active existing drafts
*-----
* ...
endmethod.

```

Edit

Similar for the EDIT action.

```

cl_i_dr_sgbt_nte_cds_demothp

method /bobf/if_frwr_draft~create_draft_for_active_entity.
* ...
*-----
*      Begin of integration of notes reuse component to create drafts for active notes
*-----

      data(lo_nte_draft_mgr) = cl_gbt_nte_actn_delgatn_mgr=>create_instance(
                                iv_objectnodetype = <ls_active>-objectnodetype
                                iv_objectid       = <ls_active>-objectid
                                iv_draftuid       = <ls_key_link>-draft
                                it_notetypes      = lt_ntypes ).

      lo_nte_draft_mgr->edit( importing et_failed_key = et_failed_active_key
                                changing co_message = eo_message ).
*-----
*      End of integration of notes reuse component to create drafts for active notes
*-----
* ...
endmethod.

```

Delete/Discard

To delegate the action DELETE and DISCARD you have to create a determination.

```

cl_i_d_delete_entity_discard_d

method /bobf/if_frwr_determination~execute.
* ...
*   determine if we are on delete or discard mode
      data(lo_draft_util) = /bobf/cl_lib_draft_active=>get_instance( iv_bo_key = if_i_sgbt_nte_cds_demothp_c=>sc_bo_key ).
      lo_draft_util->separate_keys( exporting iv_node_key   = if_i_sgbt_nte_cds_demothp_c=>sc_node-i_sgbt_nte_cds_demothp
                                it_key                 = it_key
                                importing et_draft_key   = data(lt_draft_keys)
                                et_active_key           = data(lt_active_keys)
                                et_failed_key           = data(lt_failed_keys) ).

      if lt_failed_keys is not initial.
        et_failed_key = lt_failed_keys.
        return.
      elseif lt_active_keys is not initial.
*       we are in delete mode
        lv_mode = gc_mode_delete.
      elseif lt_draft_keys is not initial.
*       we are in discard mode
        lv_mode = gc_mode_discard.
      endif.
*-----
*      Begin of integration of the notes reuse component to discard drafts or delete active notes
*-----

      data(lo_nte_draft_mgr) = cl_gbt_nte_actn_delgatn_mgr=>create_instance( iv_objectnodetype = <ls_data>-objectnodetype
                                iv_objectid       = <ls_data>-objectid
                                iv_draftuid       = <ls_data>-key
                                it_notetypes      = lt_ntypes ).

      if lv_mode = gc_mode_discard.
*       discard drafts related to the application object
        lo_nte_draft_mgr->discard( importing ev_failed = data(lv_discard_failed)
                                changing co_message = eo_message ).

      elseif lv_mode = gc_mode_delete and
        <ls_data>-isactiveentity is not initial.
*       delete active notes related to the application object

```

```

lo_nte_draft_mgr->delete( importing ev_failed = data(lv_delete_failed)
                        changing co_message = eo_message ).

```

```

endif.

```

```

*-----
*      End of integration of the notes reuse component to discard drafts or delete active notes
*-----

```

```

endmethod.

```

For more information 3.1.3. Backend Integration.

Integrate frontend

The reuse component can be embedded in the file manifest.json of the main application like shown at the followed picture.

The screenshot shows a portion of a `manifest.json` file with the following JSON structure:

```

{
  "sap.ui.generic.app": {
    "_version": "1.1.0",
    "pages": [
      {
        "entitySet": "C_Sgbt_Nte_Cds_Demotp",
        "component": {
          "name": "sap.suite.ui.generic.template.ListReport",
          "list": true
        },
        "pages": [
          {
            "entitySet": "C_Sgbt_Nte_Cds_Demotp",
            "component": {
              "name": "sap.suite.ui.generic.template.ObjectPage"
            },
            "embeddedComponents": {
              "notesComponent": {
                "id": "notesComponent",
                "componentUsage": "notesReuseComponent",
                "title": "NOTES",
                "settings": {
                  "objectId": "{objectId}",
                  "objectNodeType": "SampleObjectNodeType",
                  "draftUUID": "{DraftUUID}",
                  "noteTypes": ["NOTETYPE1", "NOTETYPE2"]
                }
              }
            }
          }
        ]
      }
    ]
  }
}

```

Annotations on the right side of the image explain the fields:

- Logical Name and the ID must be identical** (points to `id` and `name` in the `notesComponent` object).
- Documentation: [Link](#)** (points to the `notesComponent` object).
- Name of the Notes Reuse Component, which was defined in the sap.ui5/componentUsages section.** (points to `componentUsage`).
- Description in ObjectPageSection** (points to `title`).
- ID for one object of the application (see [Object ID](#))** (points to `objectId`).
- Type of one object of the application** (points to `objectNodeType`).
- Draft ID of Draft for one application object** (points to `draftUUID`).
- List of [note types](#) that shall be displayed** (points to `noteTypes`).

For more details please have a look at the documentation 3.1.2 UI Integration.

Don't forget to declare the dependency in the manifest.json to `sap.nw.core.gbt.notes.lib.reuse.notes4smarttemplate` incase of a FioriElements App. So the annotation calls should be cached ideally.

CustomizingObject

At profile RTE_CUSTOM all formatting options are disabled by default. To enable the formatting you can pass the notetype with additional information in JSON format.

The information are passed from the Notes Component to the RichTextEditor. For more information regarding these RichTextEditor settings please have a look at the [SAPUI5 SDK <https://sapui5.hana.ondemand.com/#!/api/sap.ui.richtexteditor.RichTextEditor>](https://sapui5.hana.ondemand.com/#!/api/sap.ui.richtexteditor.RichTextEditor).

Followed you can see all settings which can be passed and are supported for the RichTextEditor:

```

"noteTypes": [{
  "name": "NOTETYPE1",
  "settings": {
    "content": {
      "richTextEditor": {
        "height": "30rem",
        "showGroupFont": false,
        "showGroupFontStyle": false,
        "showGroupInsert": false,
        "showGroupLink": false,
        "showGroupStructure": false,
        "showGroupTextAlign": false,
        "showGroupUndo": false,
        "showGroupClipboard": false
      }
    }
  }
}]

```

For more details please have a look at the documentation [3.1.3 CustomizingObject](#).

No labels